Inferring Coercions for Monadic Programming

This paper shows how to use subtyping to infer uses of the basic monad operations.

CCS Concepts: • **Software and its engineering** → **Functional languages**; *Recursion.*

Additional Key Words and Phrases: strong functional programming, type-based termination, Mendler-style recursion

ACM Reference Format:
. 2026. Inferring Coercions for Monadic Programming. *J. ACM* 37, 4, Article 111 (August 2026), 2 pages. https://doi.org/XXXXXXX.XXXXXXX

1 INTRODUCTION

The basic subtyping axioms for applicatives are:

$$\frac{}{\mathsf{L}\ (T_1 \to T_2) <: \mathsf{L}\ T_1 \to \mathsf{L}T_2}\ \mathit{seq} \qquad \frac{}{T <: \mathsf{L}\ T}\ \mathit{pure}$$

[1]

2 A DECISION PROCEDURE

Consider this example subtyping:

$$\mathsf{N} \to \mathsf{N} \quad <: \quad \mathsf{L}\ \mathsf{N} \to \mathsf{L}\ (\mathsf{L}\ \mathsf{N})$$

This is valid for applicatives, because we may first use `fmap` (which follows from `pure` and `seq`) to add L to the domain and codomain parts of the type, and then we may use function subtyping to expose the codomain part, and add an extra L with `pure`. So the coercion of $f : \mathsf{N} \to \mathsf{N}$ would be

$$\mathit{pure} \cdot (\mathit{fmap}\ f)$$

Note that with our current set of coercions, we cannot apply the function subtyping rule at the beginning, because we do not have (contravariantly) $\mathsf{L}\ \mathsf{N} <: \mathsf{N}$. The challenge of admissibility of transitivity is to derive coercions that do not use composition. So we need some form of function subtyping rule that takes occurrences of L in the domain part of the upper bound type (T_2 in $T_1 <: T_2$) into account. To do this, we adopt a new function subtyping rule that keeps track of how many applications of the functor would need to be inserted.

We work with types T and T' in the syntax shown in Figure 1. We use N as a base type (it could be the natural numbers), and L as an example covariant functor (like the list datatype). The figure also gives the syntax for *modes* θ, which play an important role in our decision procedure for subtyping. The mode τ means a top-level position in the types, while a means any position. Figure 2 presents that procedure in the form of a proof system for deriving judgments of the form $T <:^{\theta} T' \triangleright k$, where $k \in \mathbb{N}$. An example derivation is shown in Figure 3.

Author's address:

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM 0004-5411/2026/8-ART111
https://doi.org/XXXXXXX.XXXXXXX

$$\begin{array}{llll} \textit{Types} & S, T & ::= & \mathsf{N} \mid \mathsf{L}\, T \mid T \to T' \\ \textit{Modes} & \theta & ::= & a \mid \tau \end{array}$$

Fig. 1. Syntax of types

$$\frac{}{\mathsf{N} <:^a \mathsf{N} \triangleright 0}\ \textit{refl} \qquad \frac{T_1' <:^\tau T_1 \triangleright k \quad T_2 <:^a T_2' \triangleright k' \quad k' \geq k}{T_1 \to T_2 <:^a T_1' \to T_2' \triangleright k}\ \textit{fun}$$

$$\frac{T <:^\tau T' \triangleright k}{\mathsf{L}\, T <:^\tau T' \triangleright k+1}\ \textit{seq-dom} \qquad \frac{T <:^a T' \triangleright k}{T <:^\tau T' \triangleright 0}\ \textit{reset}$$

$$\frac{T <:^a T' \triangleright k}{T <:^a \mathsf{L}\, T' \triangleright k+1}\ \textit{seq-cod} \qquad \frac{T <:^\theta T' \triangleright k}{\mathsf{L}\, T <:^\theta \mathsf{L}\, T' \triangleright k}\ \textit{cov}$$

Fig. 2. Proposed rules for applicatives

$$\dfrac{\dfrac{\dfrac{\dfrac{\dfrac{\dfrac{}{\mathsf{N} <:^a \mathsf{N} \triangleright 0}}{\mathsf{N} <:^\tau \mathsf{N} \triangleright 0} \quad \dfrac{\dfrac{}{\mathsf{N} <:^a \mathsf{N} \triangleright 0}}{\mathsf{N} <:^a \mathsf{L}\,\mathsf{N} \triangleright 1}}{\mathsf{L}\,\mathsf{N} <:^\tau \mathsf{N} \triangleright 1} \quad \dfrac{}{\mathsf{N} <:^a \mathsf{L}\,\mathsf{N} \triangleright 1}}{\mathsf{N} \to \mathsf{N} <:^a \mathsf{L}\,\mathsf{N} \to \mathsf{L}\,\mathsf{N} \triangleright 1}}{\mathsf{N} \to \mathsf{N} <:^\tau \mathsf{L}\,\mathsf{N} \to \mathsf{L}\,\mathsf{N} \triangleright 0} \quad \dfrac{\dfrac{\dfrac{}{\mathsf{N} <:^a \mathsf{N} \triangleright 0}}{\mathsf{N} <:^a \mathsf{L}\,\mathsf{N} \triangleright 1}}{\mathsf{L}\,\mathsf{N} <:^a \mathsf{L}\,(\mathsf{L}\,\mathsf{N}) \triangleright 1}}{\mathsf{L}\,(\mathsf{N} \to \mathsf{N}) <:^\tau \mathsf{L}\,\mathsf{N} \to \mathsf{L}\,\mathsf{N} \triangleright 1}}{(\mathsf{L}\,\mathsf{N} \to \mathsf{L}\,\mathsf{N}) \to \mathsf{L}\,\mathsf{N} <:^a \mathsf{L}\,(\mathsf{N} \to \mathsf{N}) \to \mathsf{L}(\mathsf{L}\mathsf{N}) \triangleright 1}$$

Fig. 3. Example derivation

To handle monads, we would replace the *seq* axiom with

$$\frac{}{T_1 \to \mathsf{L}T_2 <: \mathsf{L}\, T_1 \to \mathsf{L}T_2}\ \textit{bind}$$

Then I am wondering if we could just replace the *fun* rule of Figure 2 with this alternative:

$$\frac{T_1' <:^\tau T_1 \triangleright k \quad T_2 <:^a T_2' \triangleright k' \quad k' \geq 1}{T_1 \to T_2 <:^a T_1' \to T_2' \triangleright k}\ \textit{fun}',$$

REFERENCES

[1] Simon Peyton Jones. 2001. Tackling the awkward squad: monadic input/output, concurrency, exceptions, and foreign-language calls in Haskell. In *Engineering Theories of Software Construction*. IOS Press, 47–96. https://simon.peytonjones.org/Tackling-the-awkward-squad/